ENTERPRISE SECURITY ASSESSMENT LAB

API DISCOVERY AND REQUEST MAPPING METHODOLOGY

Endpoint discovery, Postman workflow organization, and API-aware route mapping

Field	Details
Module	Sub-Project 2 - API Security Testing Lab
Document Type	Individual API Security Methodology PDF
Phase / Test Case	Phase 2 - API Discovery and Mapping
Prepared For	GitHub Portfolio Documentation
Prepared By	Jagriti Banerjee
Repository	Enterprise-Security-Assessment-Lab
GitHub Filename	02-api-discovery-and-mapping-methodology-report.pdf

This document is a professional, evidence-driven explanation of one API security methodology section. It is designed for portfolio review and contains only authorized lab-based testing context. Sensitive token values, account data, cookies, and private identifiers are redacted or intentionally represented with placeholders.

1. Section Overview

Area	Details
Phase	Phase 2 - API Discovery and Mapping
Objective	Identify visible and hidden API endpoints and convert discovered requests into a structured, repeatable testing workflow.
Primary Result	Evidence captured and methodology documented
GitHub Folder	02-api-security/reports/methodology/

Why this section matters

This section demonstrates more than simple endpoint scanning. It shows a professional approach to API mapping: discovery results are organized into a request collection, making later authentication, authorization, and data exposure tests reproducible.

2. Evidence Embedded in This PDF

Evidence 1: 04-postman-collection-structure.png

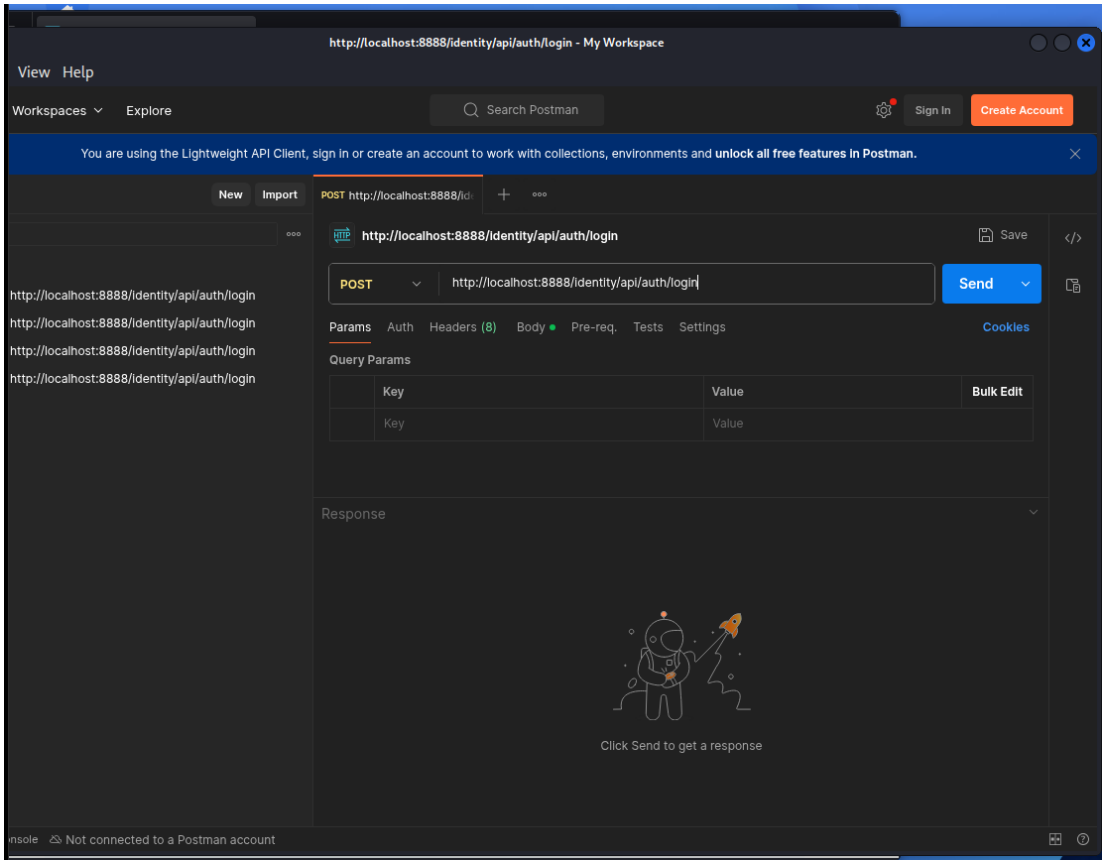

Figure 1 - Postman collection evidence showing organized API requests and repeatable workflow.

Postman collection evidence showing organized API requests and repeatable workflow.

Evidence 2:
10-kiterunner-hidden-endpoints-discovered.png

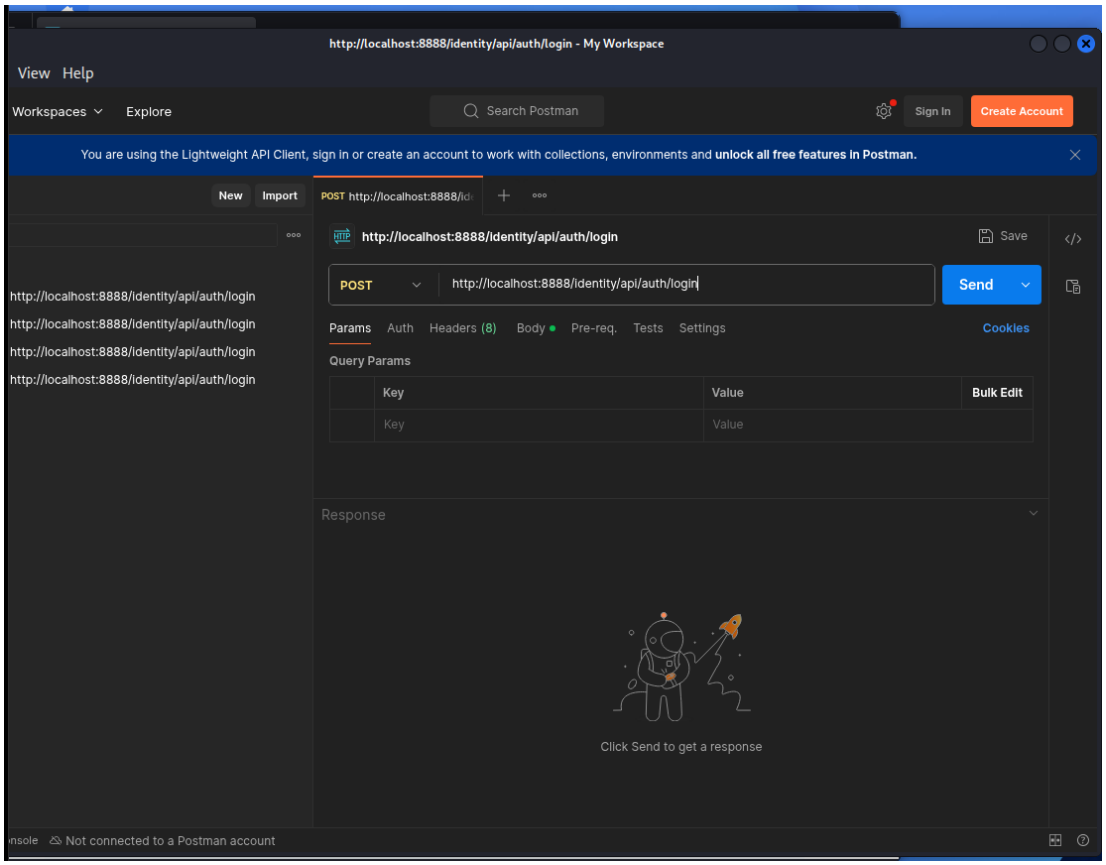

Figure 2 - Kiterunner evidence showing API-aware endpoint discovery against the lab target.

Kiterunner evidence showing API-aware endpoint discovery against the lab target.

3. Command and Workflow Used

```
# API-aware discovery using Kiterunner
kr scan http://127.0.0.1:8888

# Additional endpoint behaviour testing with ffuf
ffuf -u http://127.0.0.1:8888/FUZZ \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/api/api-endpoints.txt

# Organize discovered routes inside Postman
# Create folders: Auth, User, Vehicle, Workshop, GraphQL, OAuth, Negative Tests
```

Step-by-step workflow

- Performed API-aware endpoint discovery using Kiterunner.
- Used ffuf for additional route behaviour testing and status-code observation.
- Transferred useful endpoints into a structured Postman collection.
- Grouped requests by functionality to support repeatable testing and reporting.

4. Professional Interpretation

This section demonstrates more than simple endpoint scanning. It shows a professional approach to API mapping: discovery results are organized into a request collection, making later authentication, authorization, and data exposure tests reproducible.

5. Security Risk and Impact

Hidden or undocumented API endpoints may expose deprecated functionality, weak authorization checks, admin functions, or legacy logic. The risk increases when routes are not tracked in an API inventory or when security controls are applied inconsistently across documented and undocumented endpoints.

6. Recommended Remediation and Hardening

- Maintain a complete API inventory with all active routes and versions.
- Remove deprecated endpoints that are no longer required.
- Apply authentication and authorization middleware consistently across every route.
- Monitor unusual endpoint access and high-volume route discovery behaviour.
- Review API gateway logs for routes not used by normal clients.

7. Framework Mapping

Framework Area	Mapping
OWASP API	API9: Improper Inventory Management
MITRE-style Concept	Endpoint discovery and service mapping
Evidence Result	Testing workflow and endpoint discovery

8. Sanitized Output Style for GitHub

When documenting this evidence publicly, use placeholders instead of live values. The goal is to prove methodology and security understanding without exposing secrets or private data.

```
Authorization: Bearer <REDACTED_TOKEN>
Cookie: <REDACTED_COOKIE>
email: lab-user@example.com
password: REDACTED
object_id: <CONTROLLED_LAB_ID>
redirect_uri: <REDACTED_OR_LAB_CALLBACK>
```

9. GitHub Redaction Checklist

- Bearer tokens and JWT values must be blurred or replaced with <REDACTED_TOKEN>.
- Email addresses, phone numbers, user IDs, and object identifiers should be blurred when they are not required for understanding the evidence.
- Authorization headers, cookies, session values, OAuth codes, refresh tokens, access tokens, and client secrets must never be visible in public GitHub screenshots.

- If a screenshot contains personally identifiable data, crop the view or blur the affected rows before publication.

10. Publication Note

This report is designed for GitHub portfolio use. It describes authorized lab testing only and avoids production claims. The embedded screenshots have been treated as lab evidence and should remain redacted before upload.